

Competitive Programming

Section 25: Interval DP

Competitive Programming Course

2026-07-06

Table of contents

1	Why interval DP is different	3
2	Recognition	4
3	Length order	5
4	Pattern 1: split intervals	6
5	Pattern 2: endpoint decisions	7
6	Pattern 3: choose the last operation	8
7	Worked example: merging piles	9
8	Worked example: palindrome repair	10
9	Prefix sums for interval costs	11
10	Local exercises	12
10.1	A. Merge Piles	12
10.2	B. Palindrome Repairs	12
10.3	C. Treasure Balloons	12
11	C++ implementation notes	13
12	Python implementation notes	14
13	Proof habits	15
14	Common mistakes	16
15	Local judge workflow	17
16	Submission checklist	18

1 Why interval DP is different

In one-dimensional and grid DP, the computation order was visually obvious: left to right, top to bottom. In interval DP, the state is a contiguous range:

```
dp[l][r] = answer for subarray/string interval l..r
```

The dependencies are usually smaller intervals inside $[l, r]$, so the correct order is:

```
increasing interval length
```

That order guarantees every subinterval needed by a transition has already been computed.

2 Recognition

Interval DP is likely when:

Statement signal	Typical state
operations merge adjacent groups	$dp[l][r]$
remove items from ends	$dp[l][r]$
make substring a palindrome	$dp[l][r]$
split a range into two ranges	try all k between l and r
choose the last removed object inside a range	try every last object k

The phrase “contiguous” matters. If choices are arbitrary subsets, use subset or knapsack DP instead.

3 Length order

For an array of length n :

```
for length from 1 to n:
    for l from 0 to n-length:
        r = l + length - 1
        compute dp[l][r]
```

Why this works:

```
[l, r] depends on intervals like:
[l, k], [k+1, r], [l+1, r], [l, r-1], [l+1, r-1]
```

All of those are shorter than $[l, r]$.

4 Pattern 1: split intervals

For adjacent merging:

```
dp[l][r] = minimum cost to merge piles l..r into one pile
```

The final merge combines two already-merged adjacent groups:

```
l..k and k+1..r
```

Try every split:

```
dp[l][r] = min over k:  
    dp[l][k] + dp[k+1][r] + sum(l..r)
```

The interval sum is the cost of the final merge.

5 Pattern 2: endpoint decisions

For palindrome repair:

```
dp[l][r] = minimum insertions to make s[l..r] a palindrome
```

If endpoints already match:

```
dp[l][r] = dp[l+1][r-1]
```

If not, insert a matching character for one side:

```
dp[l][r] = 1 + min(dp[l+1][r], dp[l][r-1])
```

This is interval DP without trying every split.

6 Pattern 3: choose the last operation

Some interval problems become easy if you choose the last operation inside the interval, not the first.

For Treasure Balloons, if balloon k is the last balloon popped inside $[l, r]$, then all balloons inside $[l, k-1]$ and $[k+1, r]$ are already gone. The nearest unpopped outside balloons are fixed:

```
l-1 and r+1
```

So:

```
dp[l][r] = max over k:  
    dp[l][k-1] + dp[k+1][r] + a[l-1] * a[k] * a[r+1]
```

Choosing the first popped balloon is messy because its neighbors change in ways that depend on future operations. Choosing the last popped balloon makes the outside neighbors known.

7 Worked example: merging piles

Piles:

```
10 20 30 40
```

Length 1:

```
dp[i][i] = 0
```

Length 2:

```
dp[0][1] = 30  
dp[1][2] = 50  
dp[2][3] = 70
```

Length 3:

```
dp[0][2] = min(  
    dp[0][0] + dp[1][2] + 60 = 110,  
    dp[0][1] + dp[2][2] + 60 = 90  
) = 90
```

The final answer is 190.

8 Worked example: palindrome repair

String:

```
abcda
```

Endpoints a and a match, so:

```
dp[0][4] = dp[1][3]
```

Now substring bcd has no matching endpoints. One optimal repair inserts two characters:

```
b c d -> b c d c b
```

So answer is 2.

9 Prefix sums for interval costs

When transitions repeatedly ask for `sum(1..r)`, compute prefix sums:

```
pref[0] = 0
pref[i+1] = pref[i] + a[i]
sum(1..r) = pref[r+1] - pref[1]
```

Without prefix sums, Merge Piles would accidentally become $O(n^4)$.

10 Local exercises

10.1 A. Merge Piles

State:

```
dp[l][r] = minimum cost to merge piles l..r into one group
```

Transition:

```
try the final split k
```

10.2 B. Palindrome Repairs

State:

```
dp[l][r] = minimum insertions to make s[l..r] a palindrome
```

Transition:

```
match endpoints, or repair one side
```

10.3 C. Treasure Balloons

State:

```
dp[l][r] = maximum score from popping all balloons l..r,  
           assuming l-1 and r+1 remain outside
```

Transition:

```
try the last balloon k popped in the interval
```

11 C++ implementation notes

Use `long long` for costs and scores:

```
vector<vector<long long>> dp(n, vector<long long>(n));
```

Use 0-based indexing for ordinary intervals. For sentinel-based problems, such as Treasure Balloons, 1-based indexing with sentinels can simplify formulas:

```
a[0] = a[n+1] = 1
```

12 Python implementation notes

For $O(n^3)$ interval DP, keep loops simple and constraints moderate:

```
for length in range(2, n + 1):  
    for l in range(n - length + 1):  
        r = l + length - 1
```

Avoid recursion for these local exercises. Bottom-up length order is clearer and avoids recursion-depth issues.

13 Proof habits

Most interval DP proofs use induction on interval length.

Template:

Base: intervals of length 1 are trivial.

Inductive step:

 assume all shorter intervals are correct.

 show the transition considers every possible final/endpoint/split choice.

 show every candidate uses only valid shorter interval solutions.

The phrase “final operation” is especially powerful in interval DP.

14 Common mistakes

```
Filling dp by l then r without respecting interval length.  
Forgetting base cases for length 1 or empty intervals.  
Recomputing interval sums inside the split loop.  
Using int for merge costs.  
Choosing the first operation when the last operation gives fixed boundaries.  
Off-by-one errors with inclusive intervals.
```

15 Local judge workflow

Run this section:

```
CP_TARGET=solution python3 sections/25_interval_dp/check.py
```

Run everything:

```
CP_TARGET=solution python3 tools/check_all.py
```

16 Submission checklist

Before submitting:

```
What exactly does dp[l][r] mean?  
Are l and r inclusive?  
What are the base intervals?  
Does the transition depend only on shorter intervals?  
Do I need prefix sums?  
Am I trying split points, endpoints, or the last operation?  
Do costs require long long?
```